

Portman Platform Engineer Case (Remote)

Date/time : __/__/2026 09:00

Subject : Assignment Portfolio Data & Platform Engineer (Portman)

Introduction

Portman is Ostrica's portfolio database and model platform. Market data comes in from the vendor (LSEG) and lands in a TimescaleDB warehouse. Funds are valued and reconciled against the custodian, and the portfolio managers' models run as containers on a small Kubernetes cluster. As the platform engineer you own this chain. You keep the data complete and correct, you keep the models running without slowing down the dashboards, and you can explain to a portfolio manager why a number looks wrong.

In this case you build a small Portman of your own on synthetic data. It is designed to evaluate four things:

- data engineering and database design;
- careful handling of market and fund data;
- moving a desk model to a container without changing its results;
- the quality, testing and documentation of your work.

Logistics

- **Time allowed:** up to 72 hours from receipt of the materials. We expect about 20 to 24 hours of work. Do the parts in order and keep it simple where you must choose.
- **Submission:** a link to a Git repository or a zipped project, sent to recruitment@ostrica.nl. It must contain:
 - **Code:** a clean project that runs from a clean environment with one documented command. Include requirements.txt (or pyproject), a README and your Git history.
 - **Migrations:** versioned SQL migrations (Flyway naming V1__description.sql, or Flyway itself).
 - **Deployment:** a Dockerfile for the model and a Helm chart, deployed on a local Kubernetes cluster (k3d or kind, both free). Include the commands and the output that show it running (`kubect1 get, job logs`).
 - **Infrastructure:** Terraform for the Azure resources in Part E. It must pass `terraform validate`; you do not need an Azure subscription.
 - **CI:** a CI pipeline file (Azure DevOps YAML or GitHub Actions) with lint, type check and tests.
 - **Front end:** a small React + TypeScript page (Part E).
 - **Report:** a PDF of at most 8 pages covering your design, data issues and their treatment, results, the shadow run, and limitations.
 - **Note to the PM:** at most one page (Part E).
- **Rules:**
 - You may use any tool or library, the internet and AI tools. You must be able to explain every choice and every line in the follow-up interview. There you present your work in 20 minutes and we change one thing live.

- State your assumptions. Where something is unclear, make a reasonable choice and document it.

The starter package

All data is synthetic. Instruments, RICs, ISINs, funds, prices and custodian files are fictitious, and the funds are not Ostrica's real funds. The data contains issues, as real deliveries do: find them, decide how to treat them, and document your choices.

Folder / file	Contents
docker-compose.yml	A TimescaleDB database (PostgreSQL 16) and the mock vendor. Load the vendor image once (<code>docker load -i vendor/mock-lseg-image.tar</code>), then start both with <code>docker compose up -d</code> .
vendor/mock-lseg-image.tar	The mock market-data vendor, a simplified LSEG Data Platform, as a Docker image. Treat it as an external vendor: use only its HTTP API.
data/reference/instruments.csv	Instrument master: instrument_id, ric, isin, name, asset_type, exchange_mic, quote_currency, sector, listing and delisting date.
data/reference/identifier_history.csv	Identifier changes with validity dates.
data/reference/exchange_holidays.csv	Exchange holidays per market (MIC).
data/reference/corporate_actions.csv	Splits and cash dividends.
data/reference/funds.csv	The two funds (base currency EUR).
data/fx/fx_rates.csv	Daily FX rates: units of quote currency per 1 base currency.
data/custodian/positions.csv	Daily positions per fund, keyed by ISIN (as the custodian delivers them).
data/custodian/cash.csv	Daily cash balance and accrued fees per fund (EUR).
data/custodian/nav.csv	Every NAV delivery from the custodian, with record_type and delivered_at.
legacy_model/	The desk's Momentum Overlay model as it ran on a desktop PC (script, .bat file, notes) and the weights it produced for March to September 2026.

The vendor API

Base URL `http://localhost:8080`, app key `portman-case-key`. This is a local test value, not a secret.

- `POST /v1/session` with `{"app_key": ...}` returns a token. Send it as `Authorization: Bearer <token>`. Only one session can be active; a new session ends the old one.
- `POST /v1/datagrid` returns one date for up to 200 RICs per call. Body: `{"universe": [...], "fields": ["CLOSE_LAST", "CLOSE_OFFICIAL", "VOLUME"], "date": "YYYY-MM-DD"}`. Values are as traded (not adjusted for splits).
- `GET /v1/history?ric=...&start=...&end=...` returns one RIC for at most 31 calendar days per call. Values are split-adjusted.
- `GET /v1/usage` shows the billable calls per UTC day. It is free.
- **Limits:**
 - 1,500 billable calls per UTC day, 5 calls per second, and occasional 503 errors.
 - Both limits answer with HTTP 429. Rate limiting sends a `Retry-After` header. An exhausted quota says "daily quota exhausted" in the error body and has no `Retry-After`.
 - A call refused by the rate limit is not billable; a call that gets a 503 is.
 - The quota is shared with "the rest of the firm", so a backfill that needs more than one day of quota is a design failure, not a reason to wait.

Fields:

- **CLOSE_LAST** is the last trade of the day. Some venues publish none.
- **CLOSE_OFFICIAL** is the official closing price (the closing auction).
- **VOLUME** is the traded volume.

Definitions

Use these definitions so results are comparable. If you add other measures, explain them.

- **Valuation price:** the official close (**CLOSE_OFFICIAL**) on the valuation date. If the exchange is closed, use the last official close before it.
- **Currency:** all fund values in EUR, converted with `fx_rates.csv` on the valuation date (EUR base: divide by the rate).
- **Fund NAV:** market value of the positions + cash – accrued fees. When the custodian delivered more than one NAV for a fund and date, the latest delivery (`delivered_at`) is leading.
- **Fund return:** based on the NAV per share, not the total NAV. YTD means from 31 December 2025.
- **Price return vs total return:** total return reinvests each cash dividend at the official close on the ex-date.

Part A - Data platform (18 points)

- **A1 Schema and migrations.** Design a schema in TimescaleDB for instruments, identifiers, prices, FX, corporate actions, custodian data and model output. Deliver it as versioned migrations, each with a short reason. Use hypertables where they make sense. Explain your choices for keys and types, and for compression or retention (or why not).
- **A2 Access.** Create least-privilege roles: a loader that writes market and custodian data, a read-only role for the web app, and a role for the momentum model that can read everything but write only its own tables. Show that the model role cannot write elsewhere.
- **A3 Loading.** Load all files from `data/`, so that loading twice changes nothing.

Part B - Market-data pipeline (18 points)

- **B1 Backfill.** Load daily prices for all instruments from 1 September 2025 to 30 September 2026 through the vendor API. Stay within one day of quota, and report the billable calls from `/v1/usage`. Include the vendor's call log (see the README). Handle 401, 429 and 503 properly: retries, backoff, re-login and a clear stop when the quota is gone.
- **B2 Daily increment.** Make a daily job that loads the latest trading day(s), idempotently: running it twice changes nothing, and a corrected price overwrites the old one. Say how many calls it costs per day.
- **B3 Data quality.** Build checks for gaps, duplicates, and stale or padded values. Report what you found and what you did with it. Do not store values that were never traded or published.

Part C - Fund valuation and returns (22 points)

- **C1 Valuation.** Value both funds every NAV date from your own data, and reconcile your NAV with the custodian's official NAV. Show the difference in basis points per fund and date.
- **C2 Breaks.** Explain every difference above 1 bp: who is wrong, why, and what you would do about it. Also list the data issues you had to fix to get this far.
- **C3 Returns.** As of 30 September 2026, give the daily and YTD return of both funds. Explain why the YTD return differs from the YTD change in total NAV.

- **C4 Price vs total return.** For the ETF OHYG.L, give the YTD price return and total return in USD.

Part D - Model migration (22 points)

The desk wants the Momentum Overlay off the desktop PC and onto the cluster without any change in results. The PC outputs in `legacy_model/legacy_output/` are your reference.

- **D1 Port.** Port the model so that it reads from your database and writes its weights to its own table. It must use its own role and must not call the vendor.
- **D2 Deploy.** Package the model and run it on a local Kubernetes cluster (k3d or kind):
 - a Dockerfile (small, non-root);
 - a Helm chart with a schedule (first weekday of the month, 08:30 Amsterdam time), resource requests and limits, and the database credentials from a Kubernetes secret;
 - a statement timeout, so the model can never slow down the dashboards.

The model in the cluster must reach your database. Show it working: the scheduled object, one run started by hand, its log, and the rows it wrote. Explain your choice between a CronJob and a long-running scheduler.

- **D3 Shadow run.** Run your port for the month-ends of March to September 2026 (file `target_weights_YYYY-MM.csv` is the run for the last day of month YYYY-MM) and compare it with the PC output, month by month. Prove the results are unchanged, or explain every difference and fix the cause.

Part E - Delivery, front end and infrastructure (20 points)

- **E1 API.** A small FastAPI endpoint `GET /funds/{fund_id}/returns?date=` that returns the NAV per share and the daily and YTD return. Use the read-only role, and add tests.
- **E2 Front end.** A small React + TypeScript page that shows both funds' NAV per share and daily and YTD return from your API. Follow our front-end rules:
 - the client shows numbers and never calculates a return;
 - an unknown value is shown as "-", never as 0;
 - all formatting (for example $\times 100$ and the % sign) happens in one file.

Add tests for the formatting.

- **E3 CI and docs.** A CI pipeline with lint, type check and tests (Python and TypeScript), plus a README and a short runbook for the daily job: what to check when it fails, and how to re-run a day.
- **E4 Azure.** In production the model gets its database password from Azure Key Vault. Write Terraform for:
 - a Key Vault (RBAC mode) holding the model's database password;
 - a user-assigned managed identity for the model that can read only that one secret;
 - a federated credential, so the model's Kubernetes service account can use the identity without a stored secret;
 - an Entra ID group "Portman Desk" that may use the web app, and nobody else.

It must pass `terraform validate`. Explain why each permission is the smallest one that works.

- **E5 Note to the PM.** On 15 July 2026 the PM of the Global Equity Fund asks why "the Portman NAV" and the custodian's NAV for 14 July differ. Write a note of at most one page that answers them in plain language.

Assessment

Part	Points
A - Data platform	18
B - Market-data pipeline	18
C - Fund valuation and returns	22
D - Model migration	22
E - Delivery, front end and infrastructure	20
Total	100

We look at correctness, at care with production (idempotent, testable, documented), and at how you explain what you found. A smaller solution that is correct and well tested scores higher than a large one that is not.